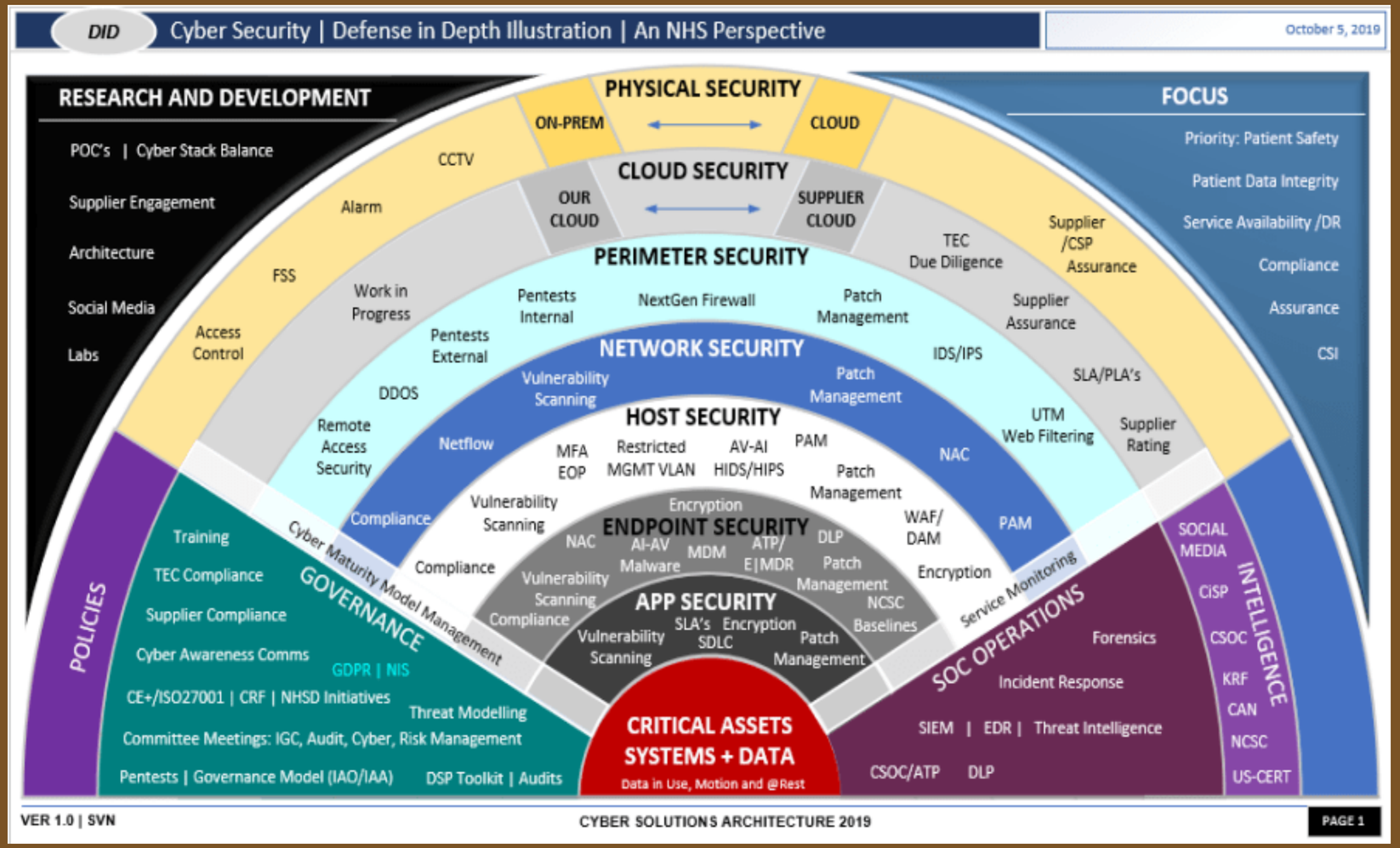
08.10.26

Crystal Schippers

Security Policy

Green Pace

DEFENSE IN DEPTH



Green Pace

OVERVIEW

PURPOSE

Standardize secure development practices across Green Pace.

SCOPE

Staff who create, deploy, or support custom software

Foundation

Defense in depth — multiple layers protect confidentiality, integrity, and availability.

Policy Areas

Secure C/C++ coding, encryption, authentication, authorization, and auditing.

Security is built into development rather than added only after an incident.

Green Pace

Threat Matrix

Highest concern: SQL injection and memory-safety failures because exploitation can expose or corrupt sensitive data.

High concern: unsafe data conversions, unbounded strings, and incorrect exception behavior.

Medium concern: iterator misuse, file-stream positioning, and weak assertions can cause undefined or unreliable behavior.

Attack Surface

Attack surface is reduced through input validation, default deny, least privilege, sanitization, testing, and secure coding standards.

Threat ratings should be reviewed as systems, dependencies, and attack techniques change.

Green Pace

Automation & External Testing

- IDE/compiler warnings provide early feedback while code is being written and built.
- Cppcheck provides independent static analysis and can identify issues such as buffer bounds, uninitialized data, exception problems, and unsafe comparisons.
- Visual Studio Code Analysis provides complementary compiler and analyzer warnings.
- Unit tests verify expected behavior and negative conditions before release.
- Defense in depth is strengthened when multiple tools independently inspect the same code.

Green Pace

Security Principles

01 Validate Input Data

02 Heed Compiler Warnings

03 Architect and Design for Security Policies

04 Keep It Simple

05 Default Deny

06 Adhere to the Principle of Least Privilege

07 Sanitize Data Sent to Other Systems

08 Practice Defense in Depth

09 Use Effective Quality Assurance Techniques

10 Adopt a Secure Coding Standard

Green Pace

Principles Mapped to Coding Standards

- Input validation/sanitization → STD-333-C, STD-444-C, STD-1010-CPP
- Compiler warnings/QA → STD-222-C, STD-666-C, STD-777-CPP, STD-888-CPP
- Security architecture/least privilege → STD-111-C, STD-444-C, STD-1010-CPP
- Defense in depth → all standards, because no single control is relied upon.
- Secure coding standards provide a repeatable baseline for developers.

Green Pace

CODING STANDARDS

Priority Order

1. STD-444-C – SQL Injection
2. STD-555-C – Memory Protection
3. STD-1010-CPP – Characters & Strings
4. STD-222-C – Data Value
5. STD-333-C – String Correctness
6. STD-777-CPP – Exceptions
7. STD-666-C – Assertions
8. STD-999-CPP – Containers
9. STD-888-CPP – Input/Output
10. STD-111-C – Data Type

Green Pace

At Rest

Protect sensitive information stored on disks, databases, backups, and other persistent media.

In Flight

Protect information while it moves between clients, applications, and services.

In Use

Protects sensitive information while it is actively processed when the technology and architecture support it.

ENCRYPTION STRATEGY

Encryption should be selected according to data sensitivity and risk, with keys protected separately from encrypted data.

Encryption is one layer of defense in depth, not a replacement for access control.

Green Pace

Triple-A Framework

Green Pace already maintains operating-system, firewall, and anti-malware logs. Triple-A supports accountability and helps detect and investigate unauthorized activity.

Accounting

Record user logins, database changes, new-user creation, access levels, and files accessed.

Authentication

Verify user identity during login and other sensitive access points.

Authorization

Enforce the user’s permitted level of access using least privilege and default deny.

Green Pace

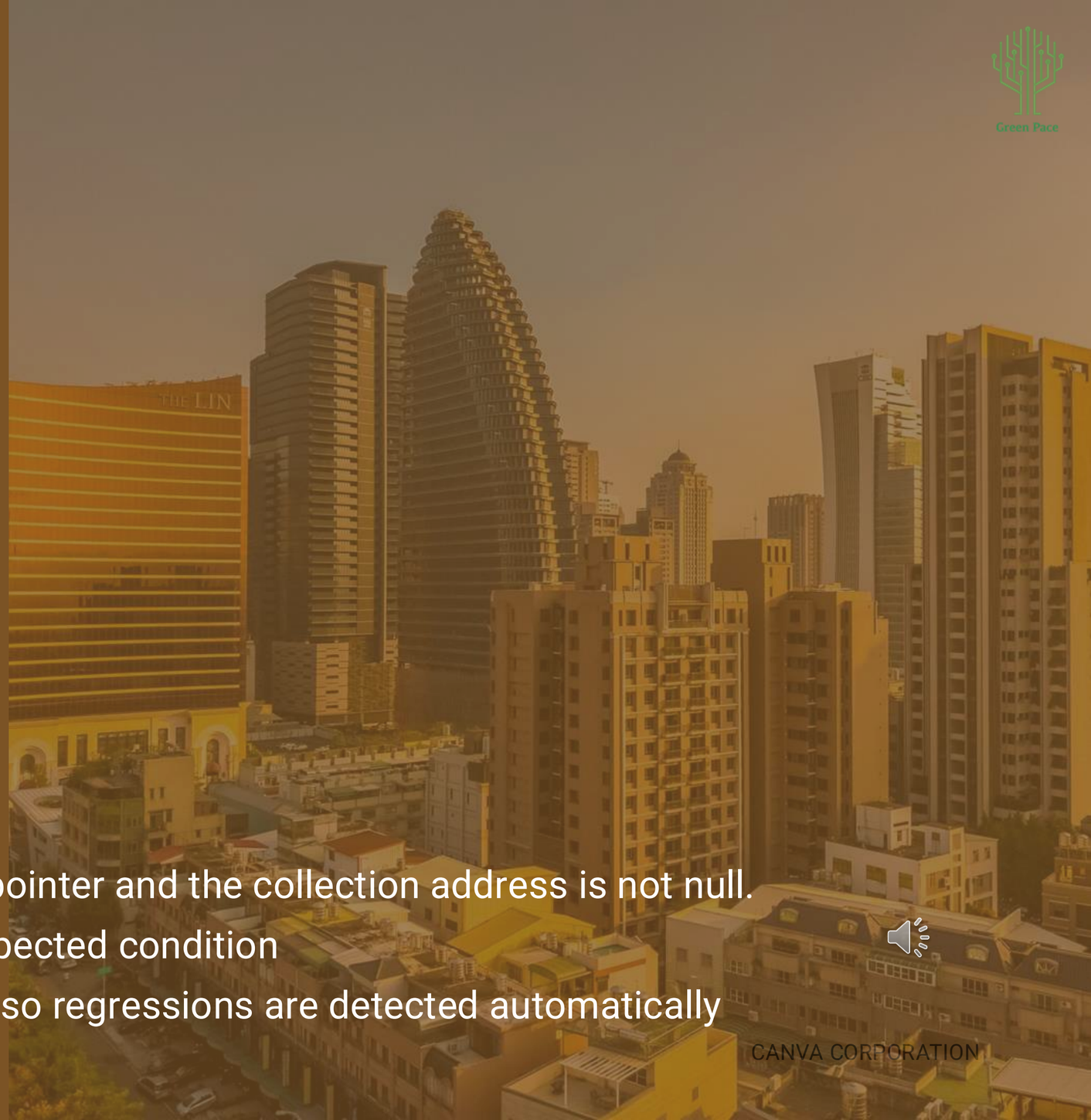
01

Unit Test:

CollectionSmartPointerIsNotNull

Test Type: Positive test

- Confirms the fixture creates a valid smart pointer and the collection address is not null.
- Google Test macros explicitly prove the expected condition
- Next Step: retain this test in the CI pipeline so regressions are detected automatically



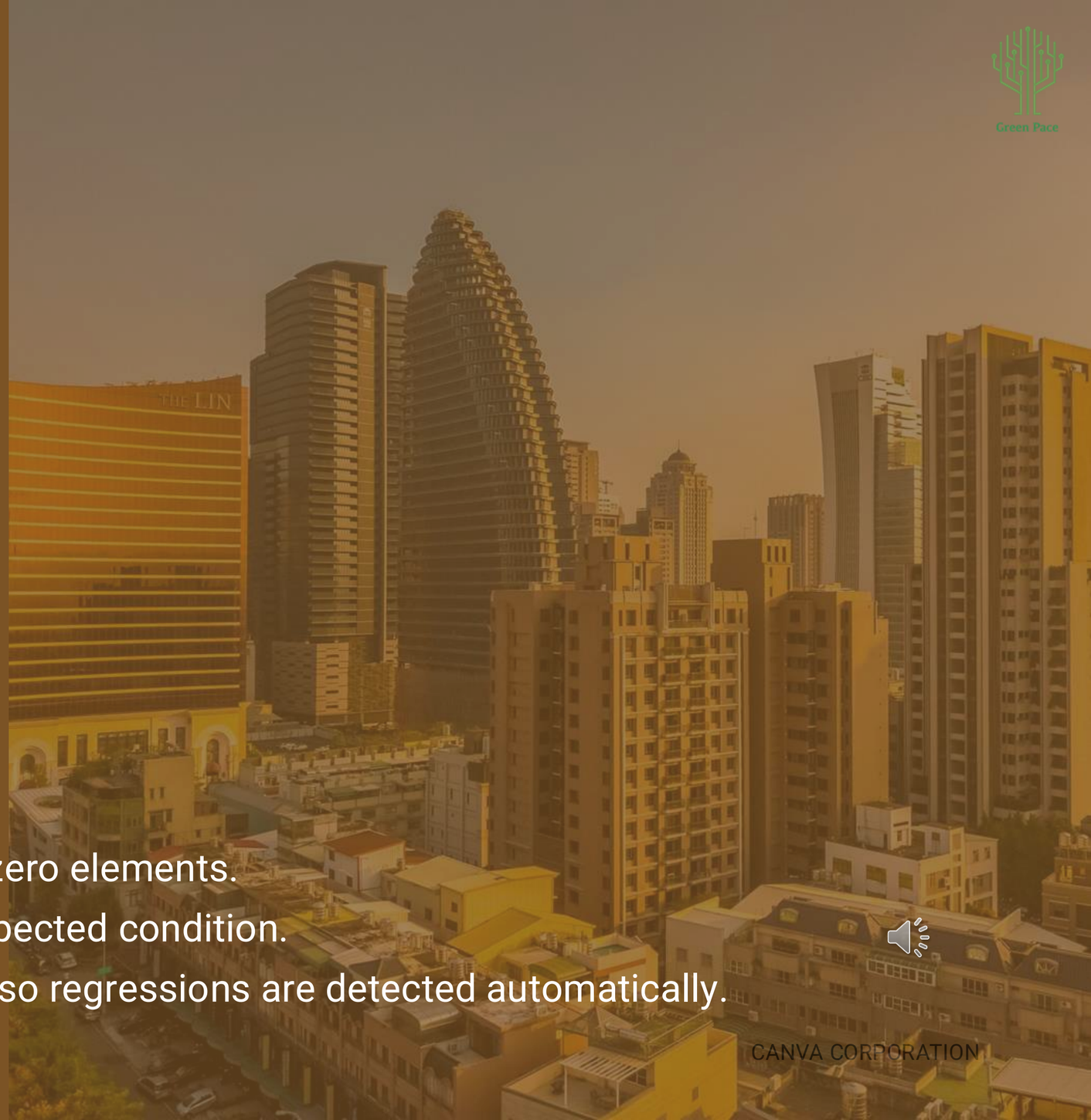
02

Unit Test:

IsEmptyOnCreate

Test Type: Positive test

- Confirms a newly created vector contains zero elements.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



Green Pace

CANVA CORPORATION

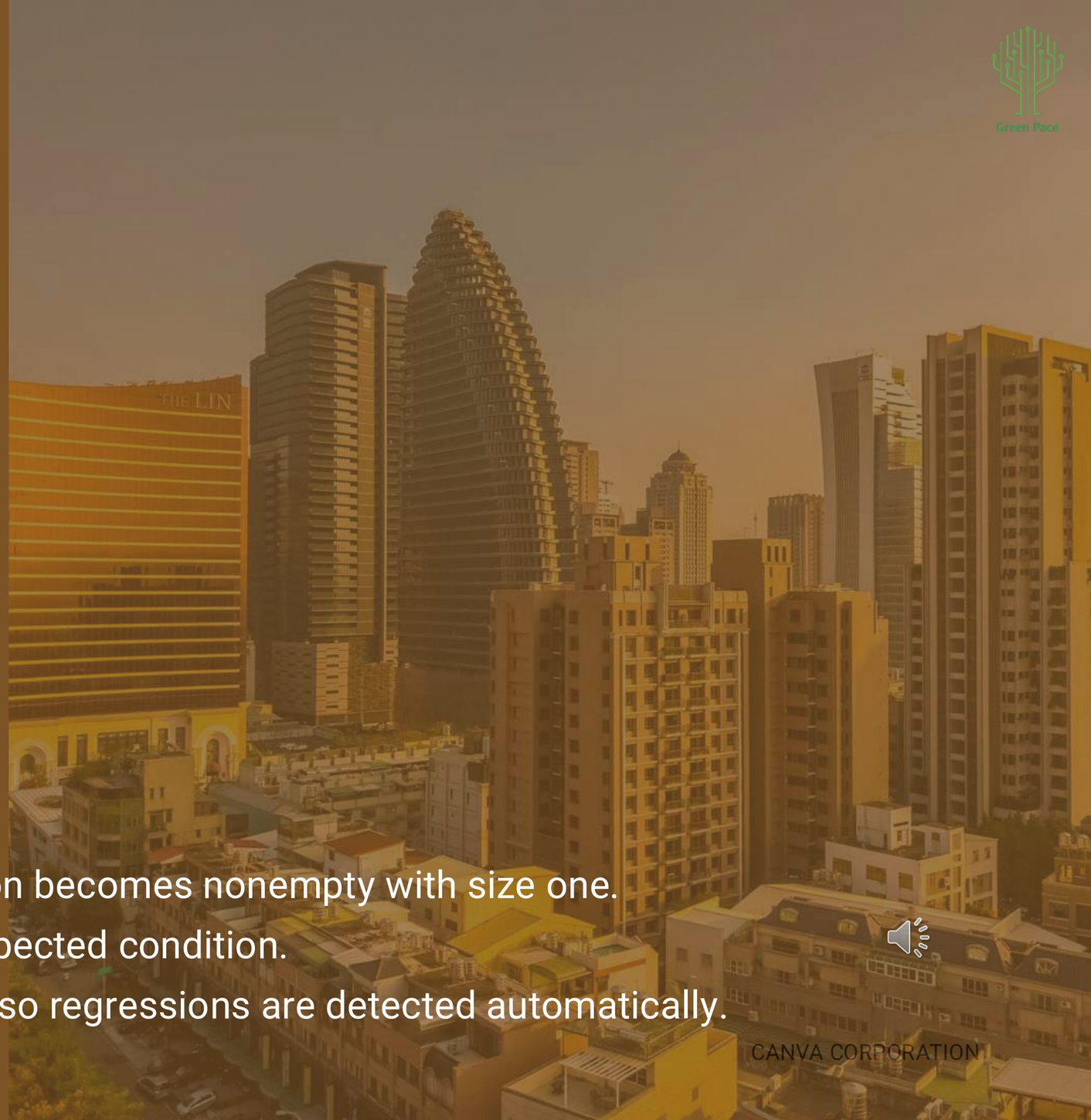
03

Unit Test:

CanAddToEmptyCollection

Test Type: Positive test

- Adds one element and verifies the collection becomes nonempty with size one.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



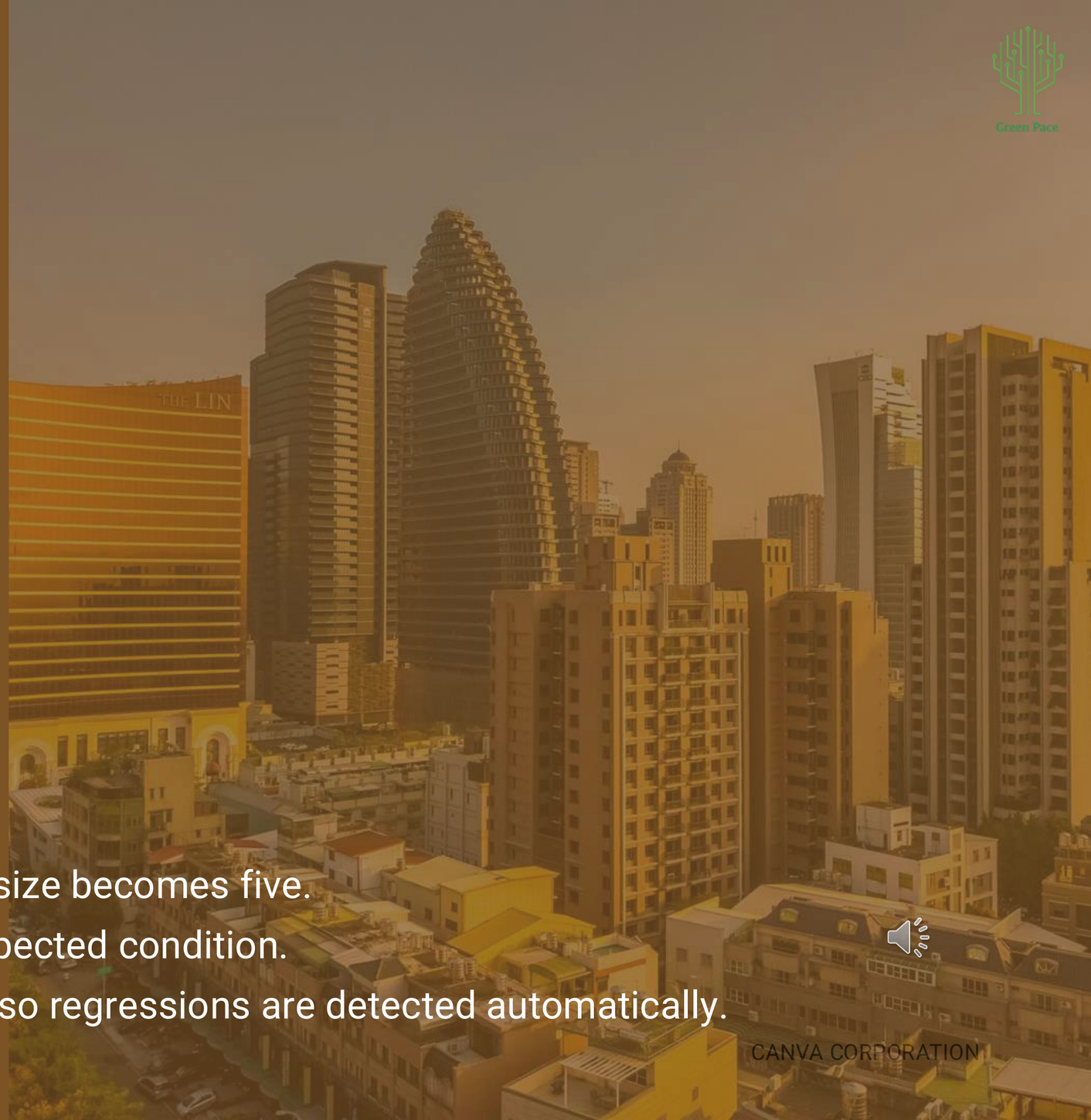
04

Unit Test:

CanAddFiveValuesToCollection

Test Type: Positive test

- Adds five elements and verifies the vector size becomes five.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



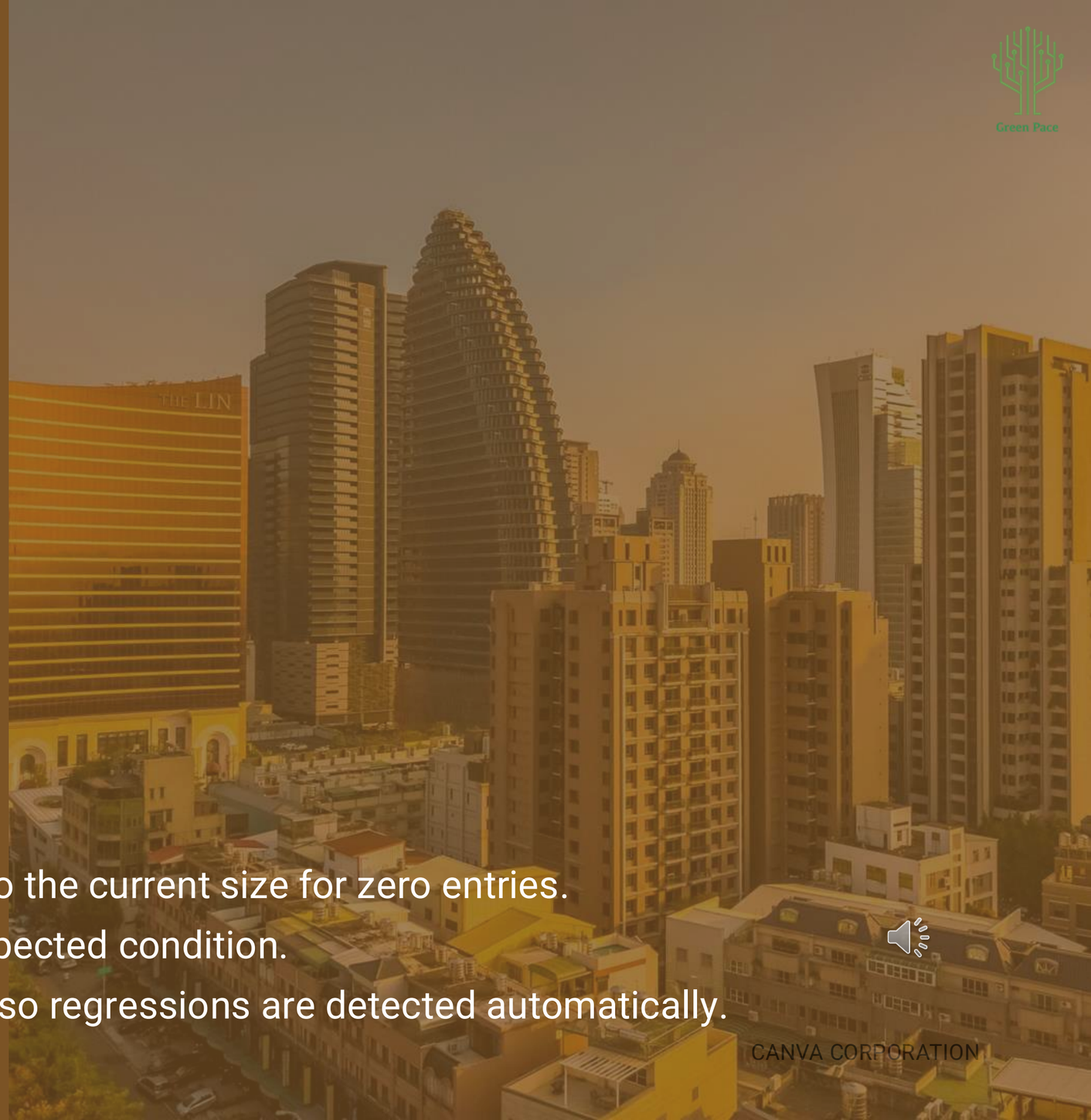
05

Unit Test:

MaxSizeGreaterThanSize(0)

Test Type: Positive test

- Verifies max_size is greater than or equal to the current size for zero entries.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



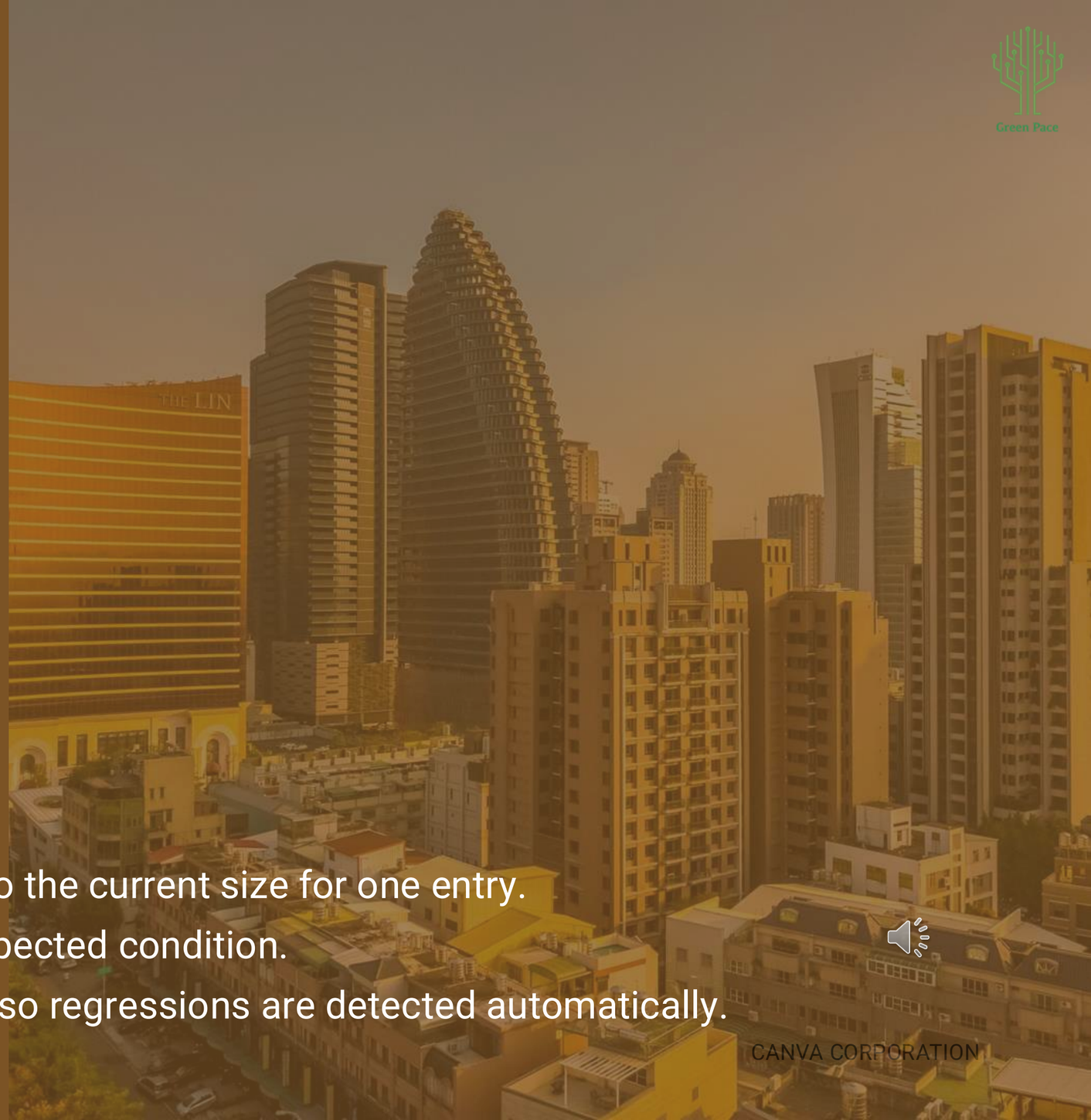
06

Unit Test:

MaxSizeGreaterThanSize(1)

Test Type: Positive test

- Verifies max_size is greater than or equal to the current size for one entry.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



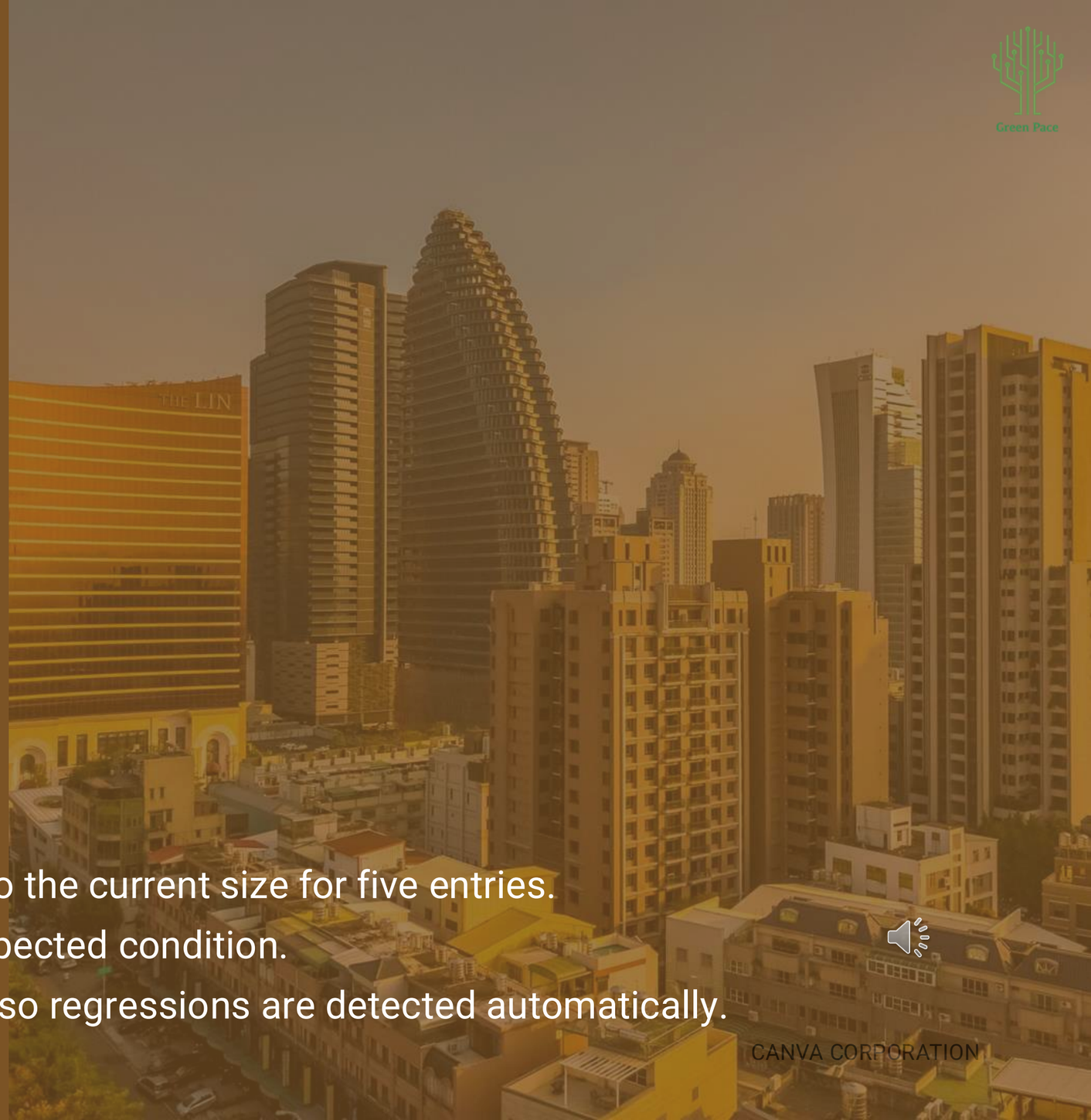
07

Unit Test:

MaxSizeGreaterThanSize(5)

Test Type: Positive test

- Verifies max_size is greater than or equal to the current size for five entries.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



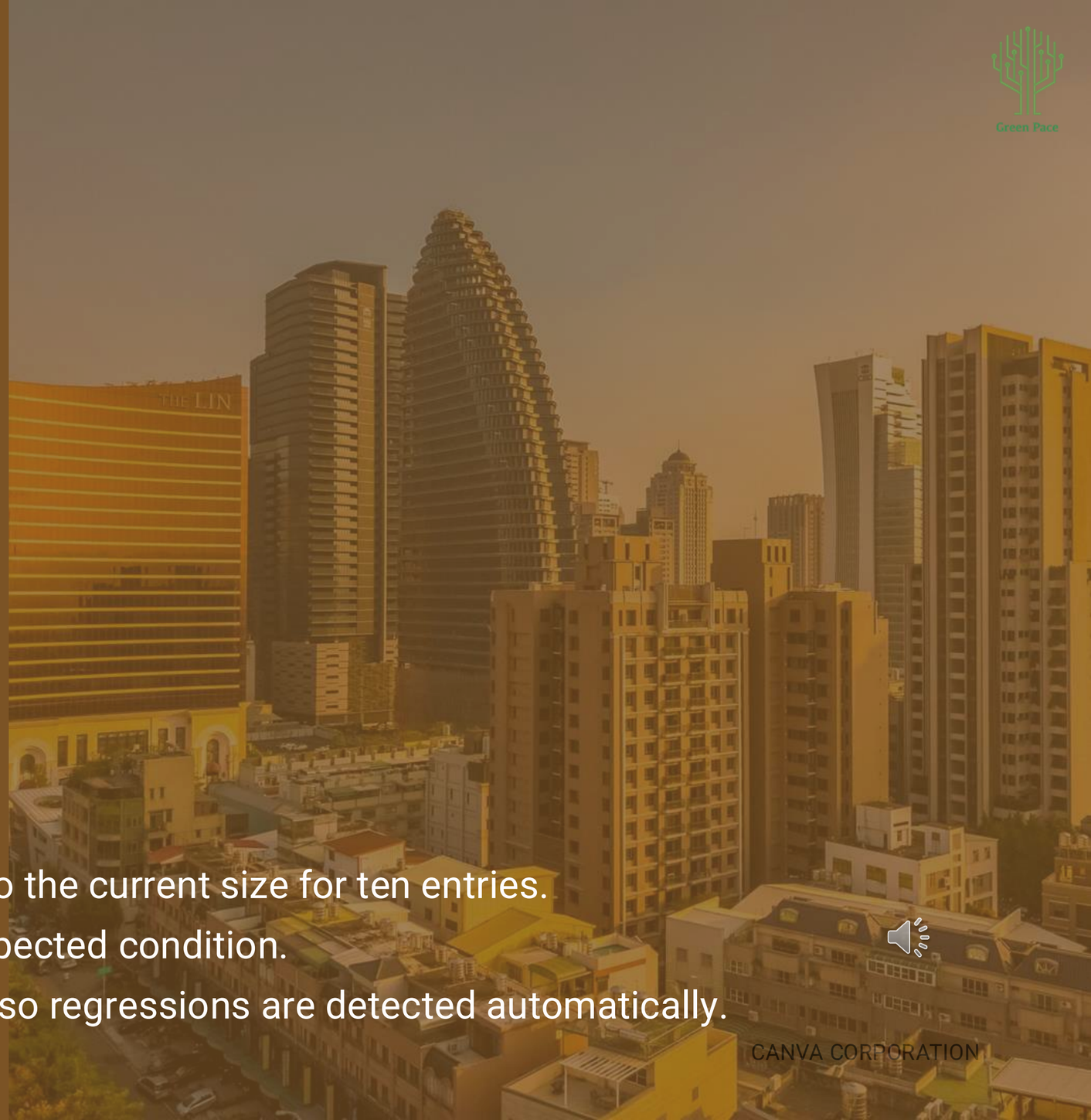
08

Unit Test:

MaxSizeGreaterThanSize(10)

Test Type: Positive test

- Verifies max_size is greater than or equal to the current size for ten entries.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



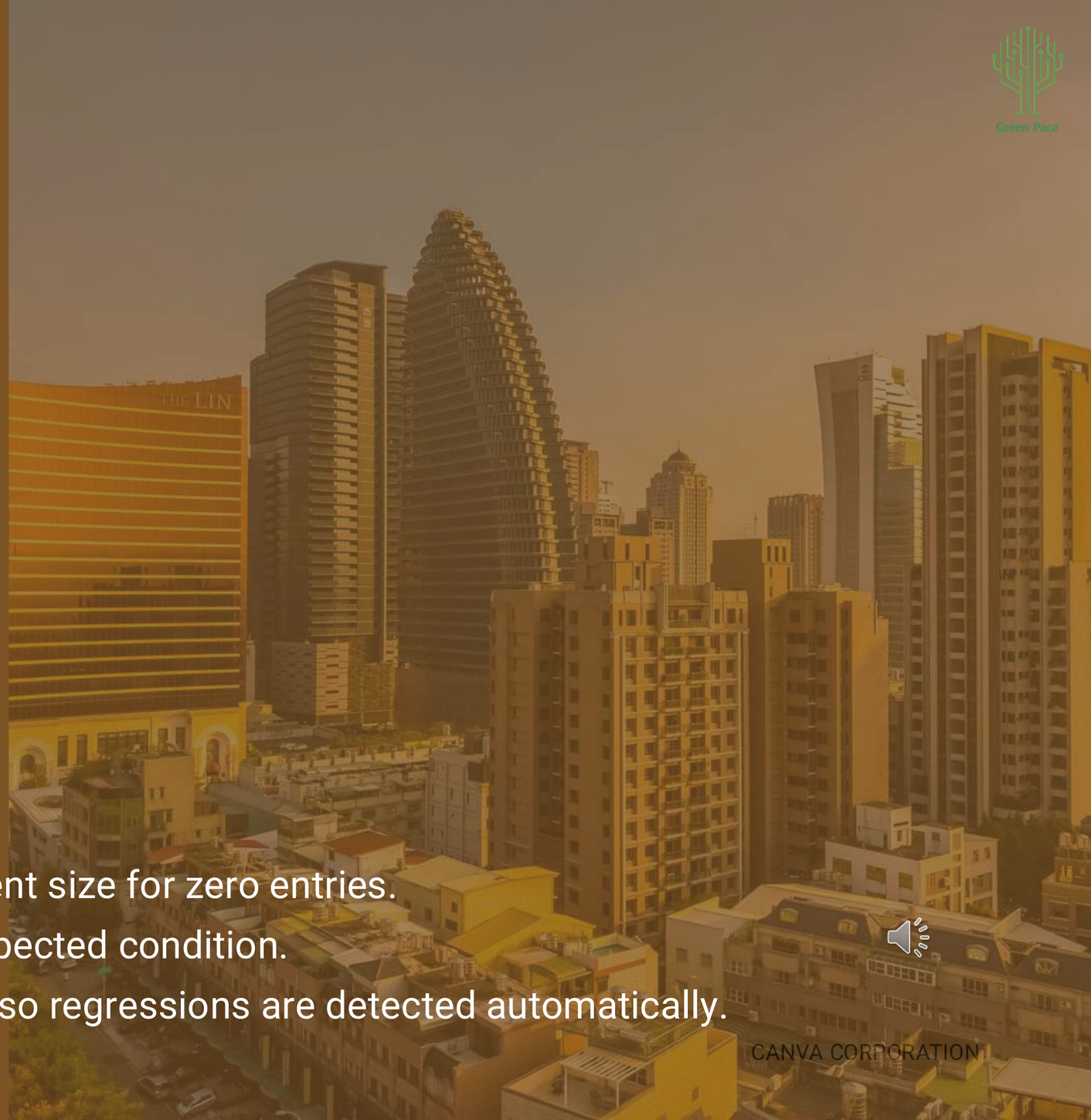
9

Unit Test:

CapcacityGreaterThanOrEqualToSize(0)

Test Type: Positive test

- Verifies vector capacity is at least the current size for zero entries.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



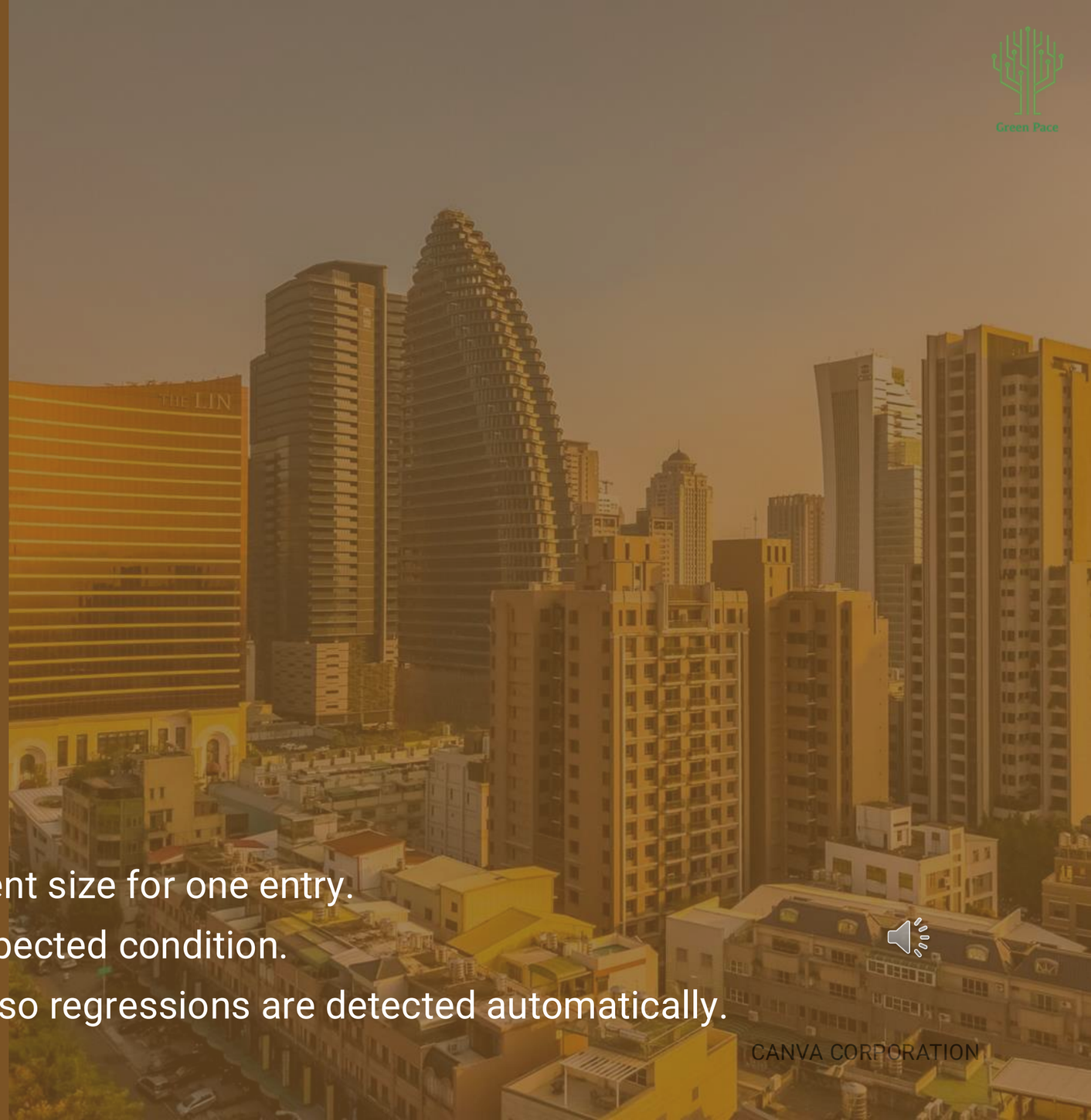
10

Unit Test:

CapcacityGreaterThanOrEqualToSize(1)

Test Type: Positive test

- Verifies vector capacity is at least the current size for one entry.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



Green Pace

CANVA CORPORATION

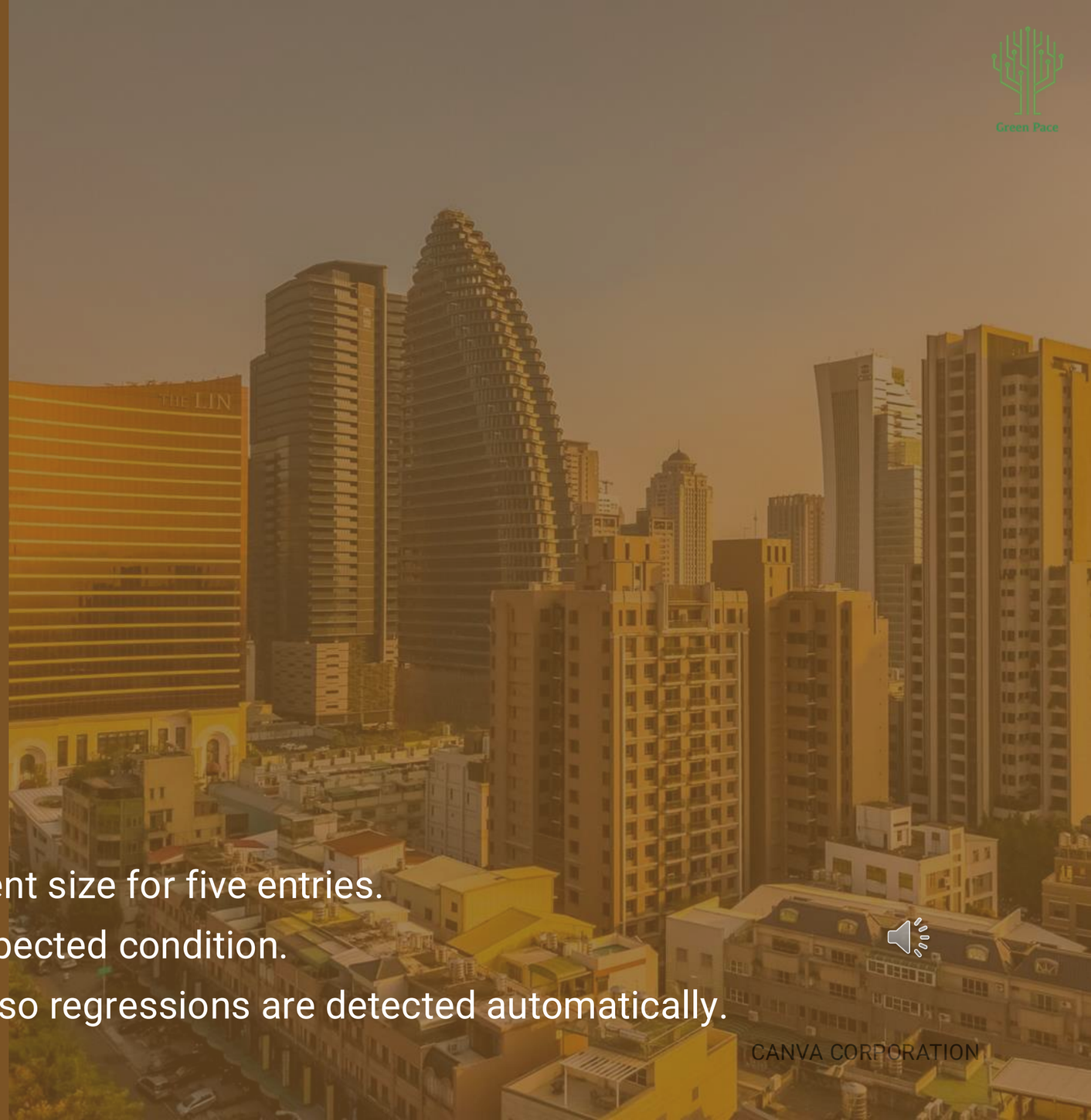
11

Unit Test:

CapcacityGreaterThanOrEqualToSize(5)

Test Type: Positive test

- Verifies vector capacity is at least the current size for five entries.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



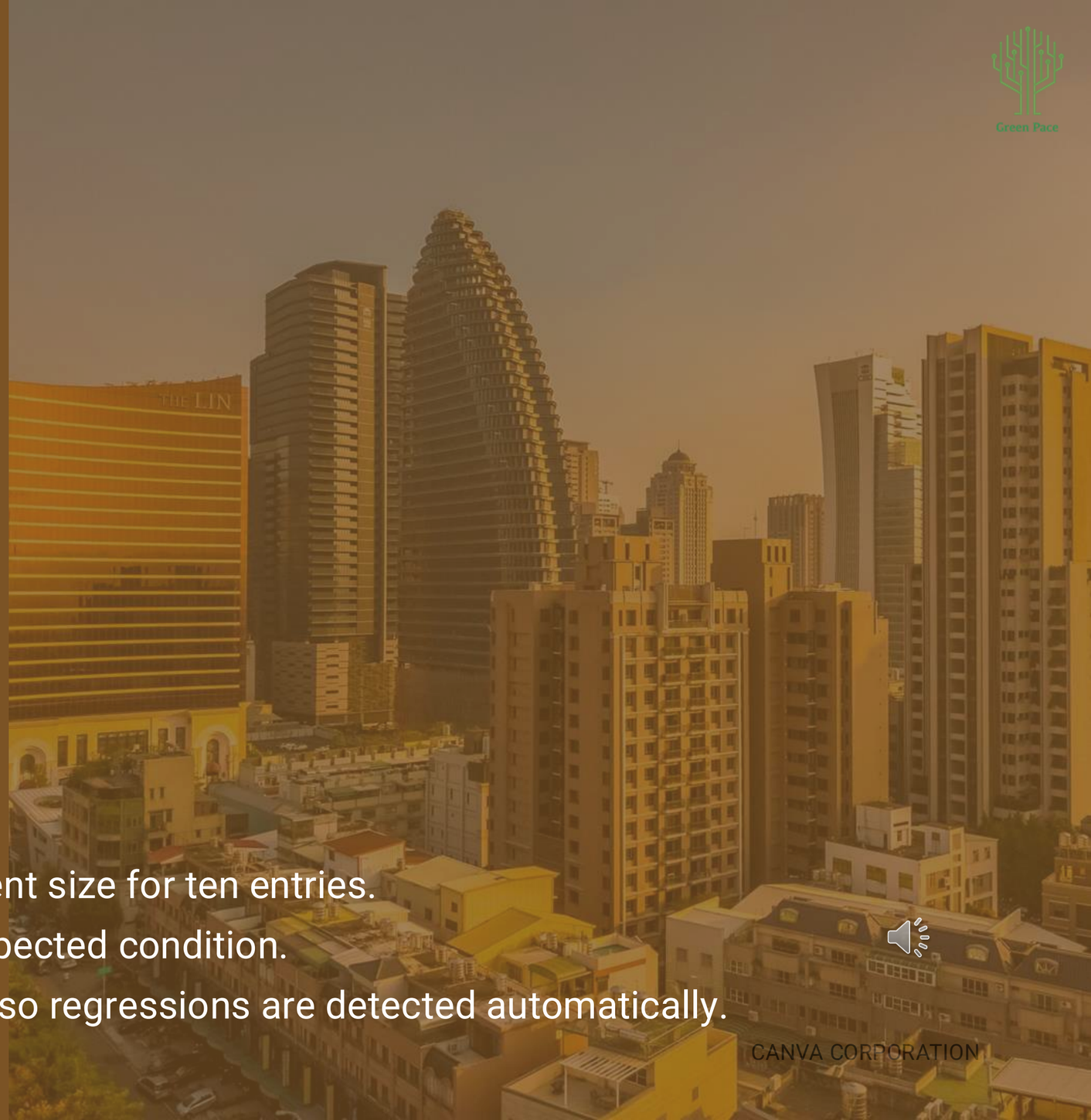
12

Unit Test:

CapcacityGreaterThanOrEqualToSize(10)

Test Type: Positive test

- Verifies vector capacity is at least the current size for ten entries.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



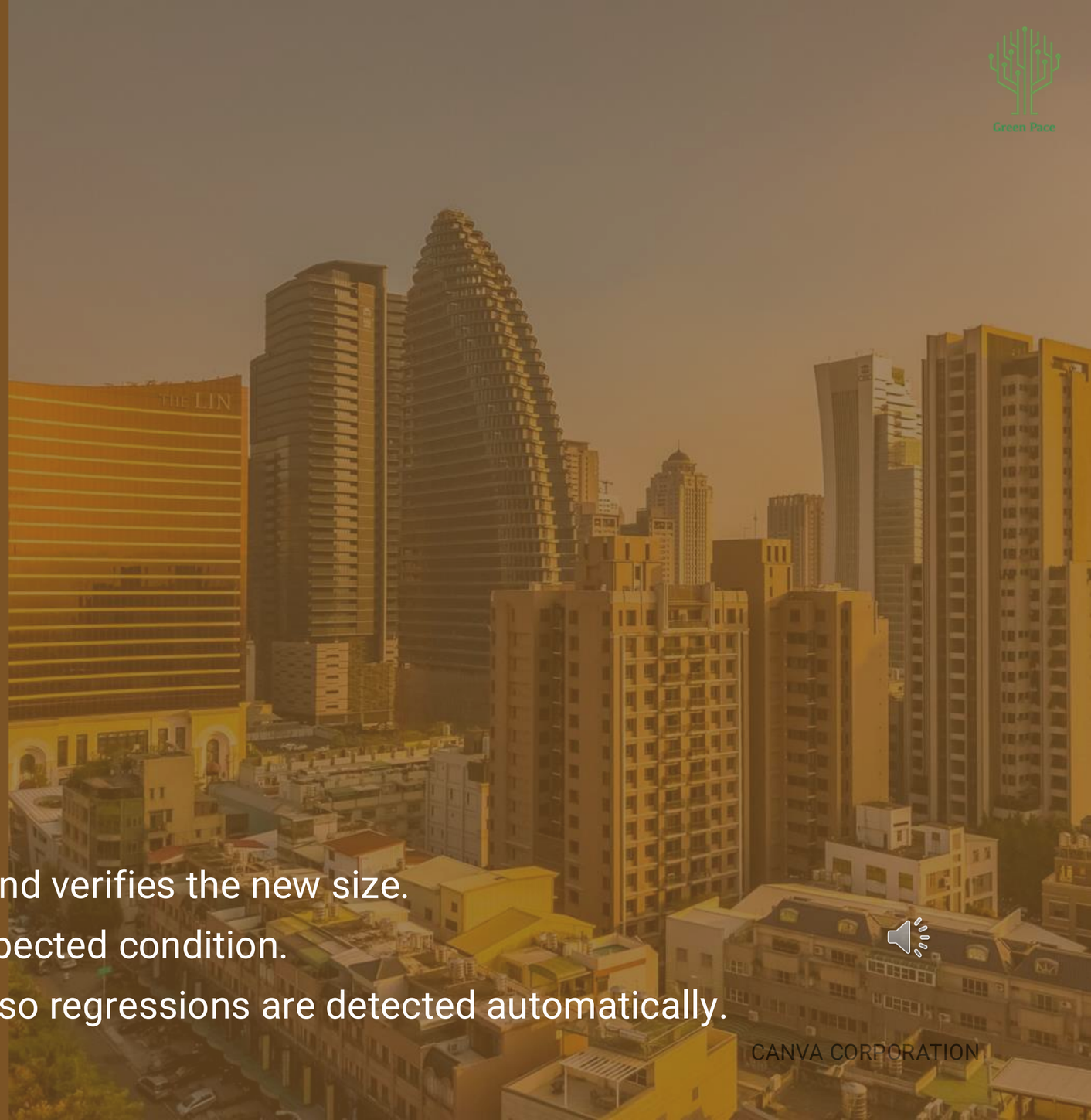
13

Unit Test:

ResizeIncreasesCollectionSize

Test Type: Positive test

- Resizes an empty vector to five elements and verifies the new size.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



Green Pace

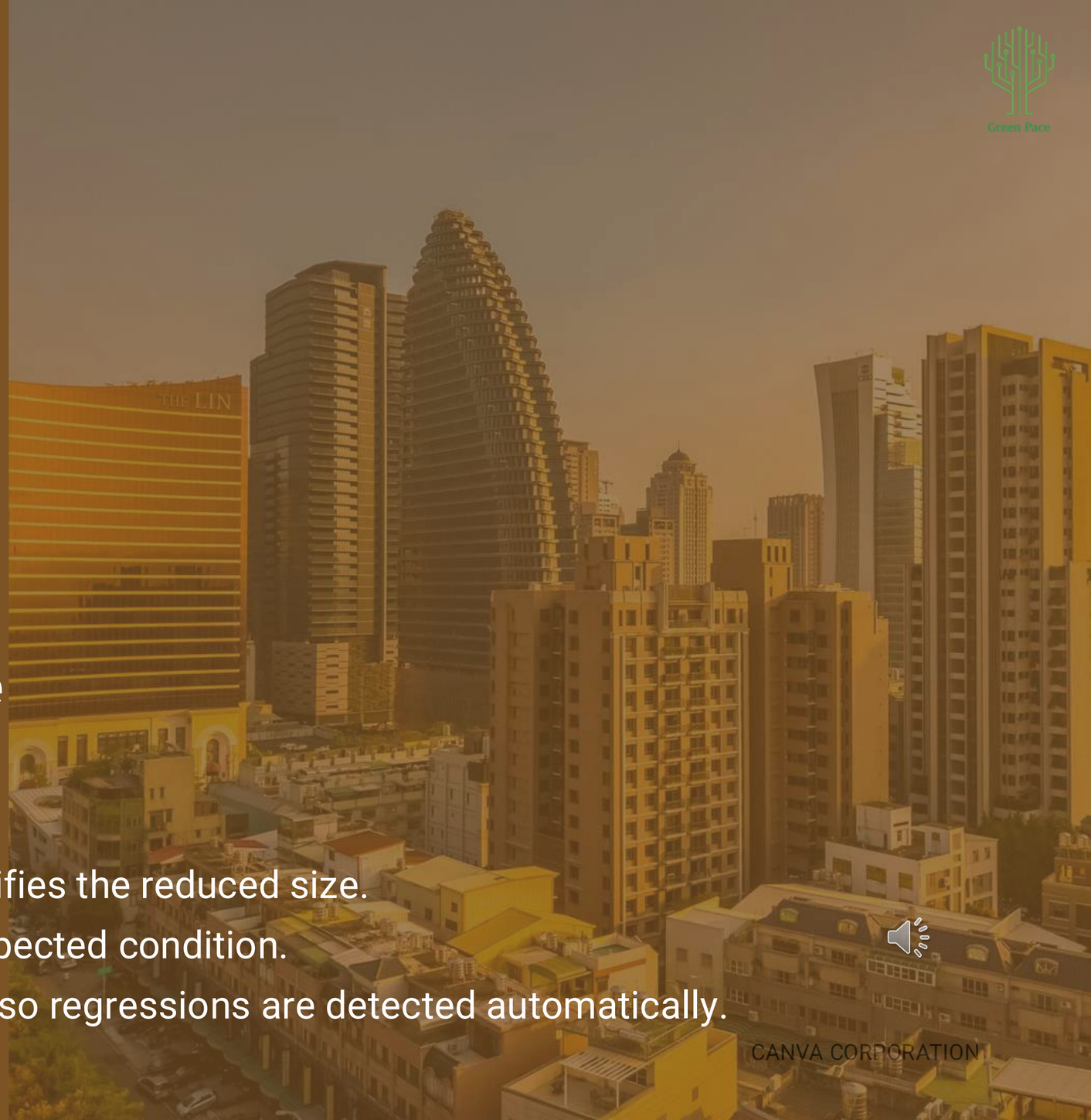
CANVA CORPORATION

14

Unit Test: ResizeDecreasesCollectionSize

Test Type: Positive test

- Adds five elements, resizes to two, and verifies the reduced size.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.

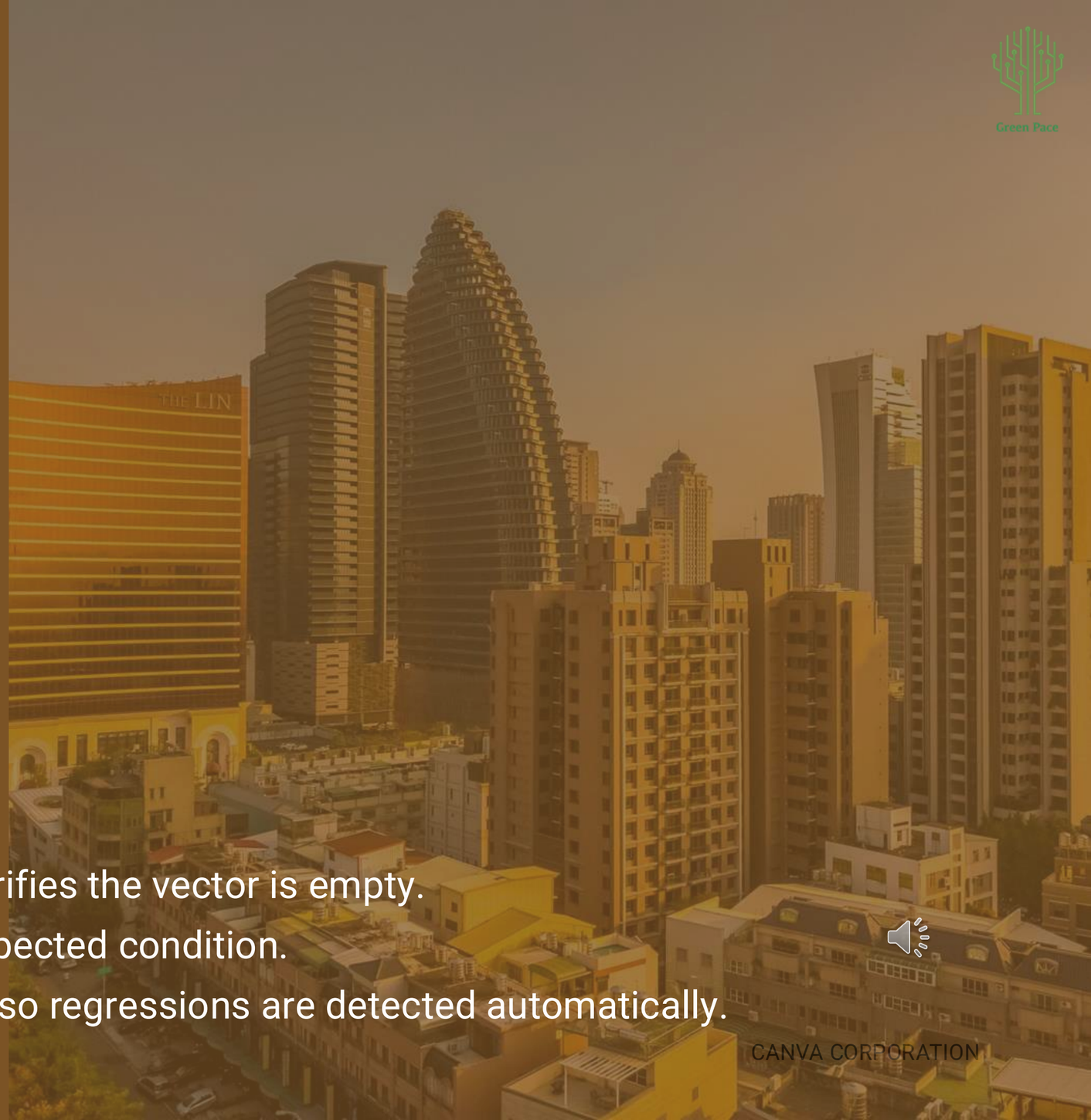


15

Unit Test: ResizeCollectionSizeToZero

Test Type: Positive test

- Adds five elements, resizes to zero, and verifies the vector is empty.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.

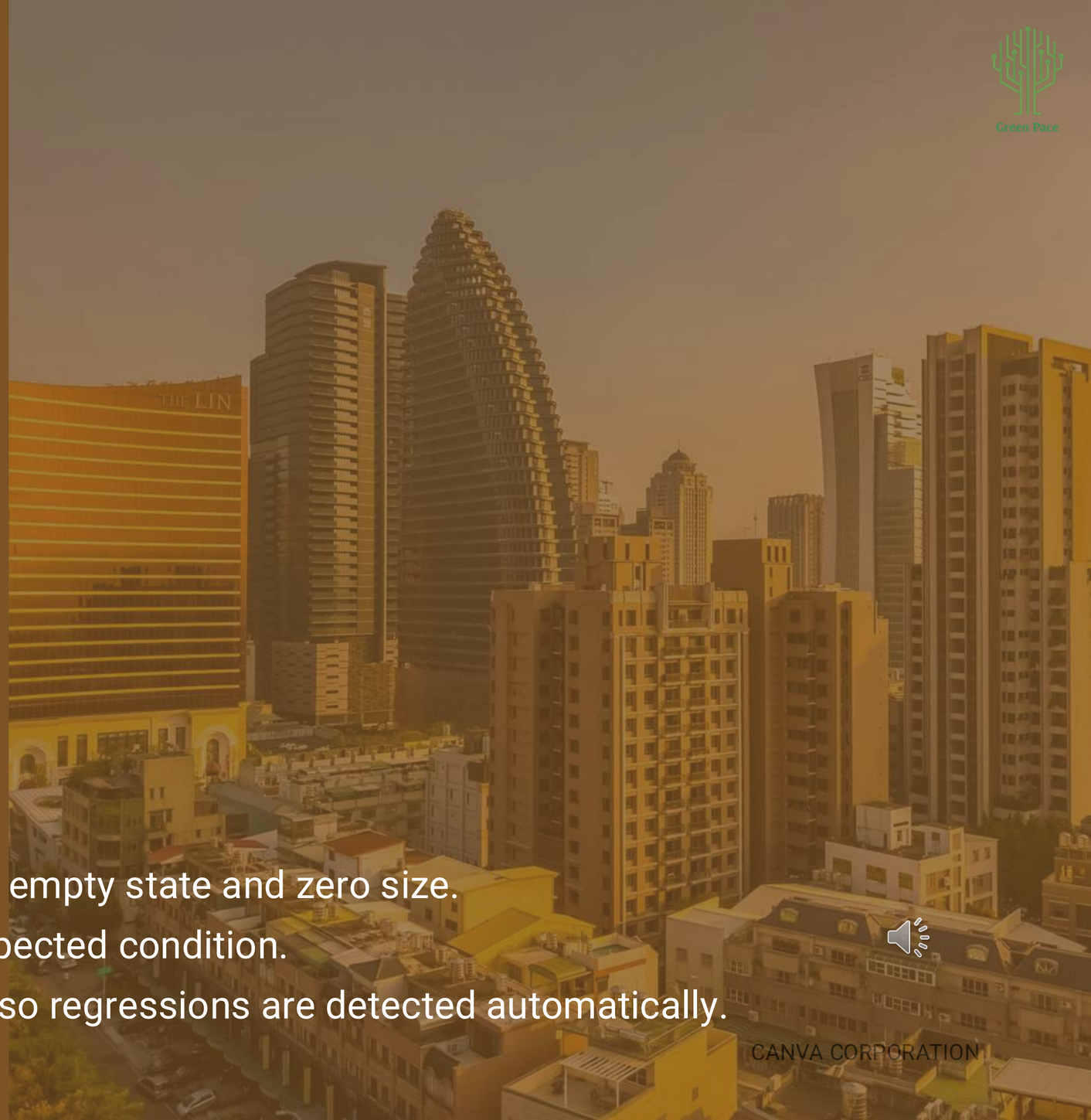


16

Unit Test: ClearCollection

Test Type: Positive test

- Clears a populated vector and verifies both empty state and zero size.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



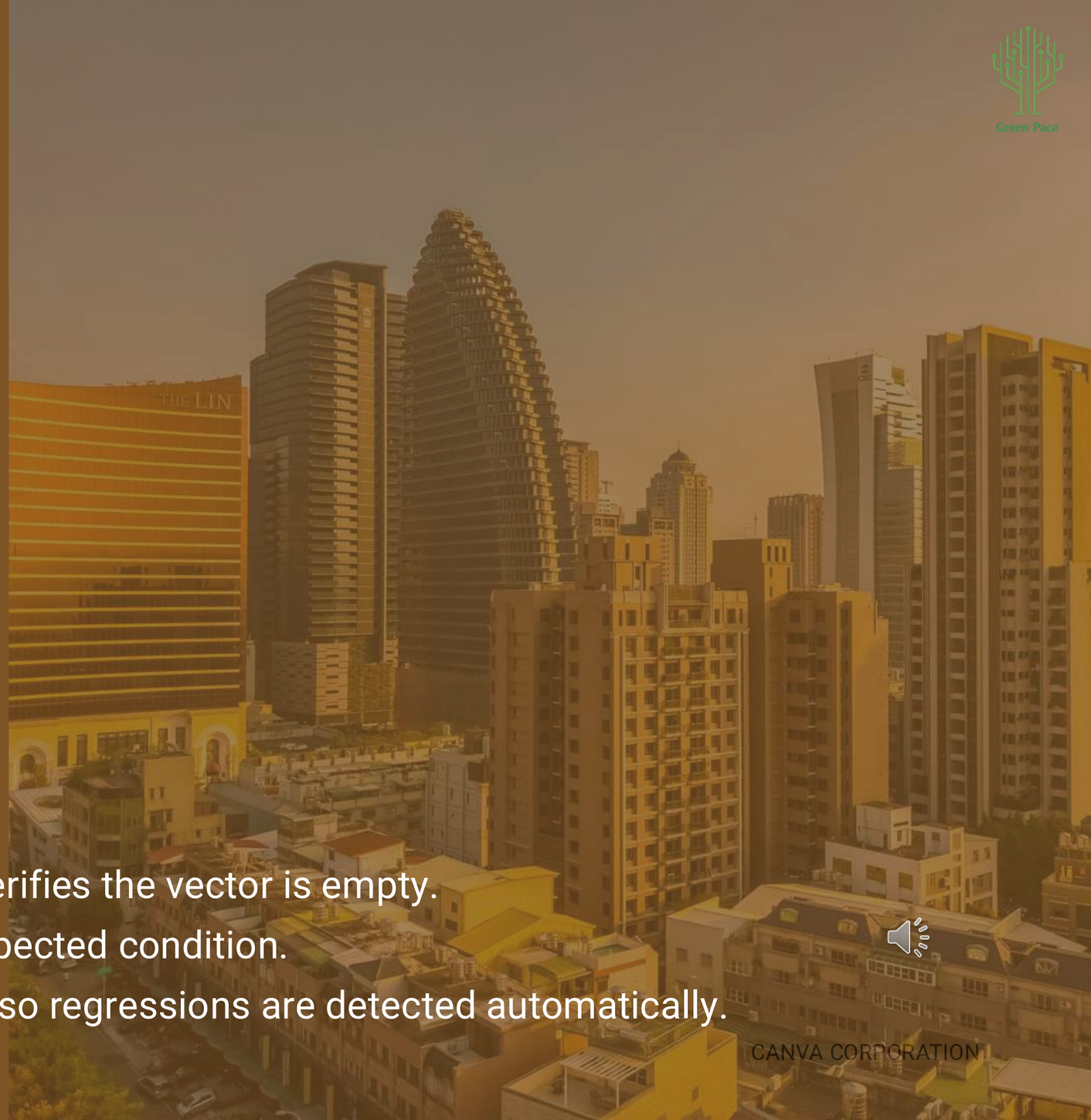
Green Pace

17

Unit Test: EraseCollection

Test Type: Positive test

- Erases the entire begin-to-end range and verifies the vector is empty.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



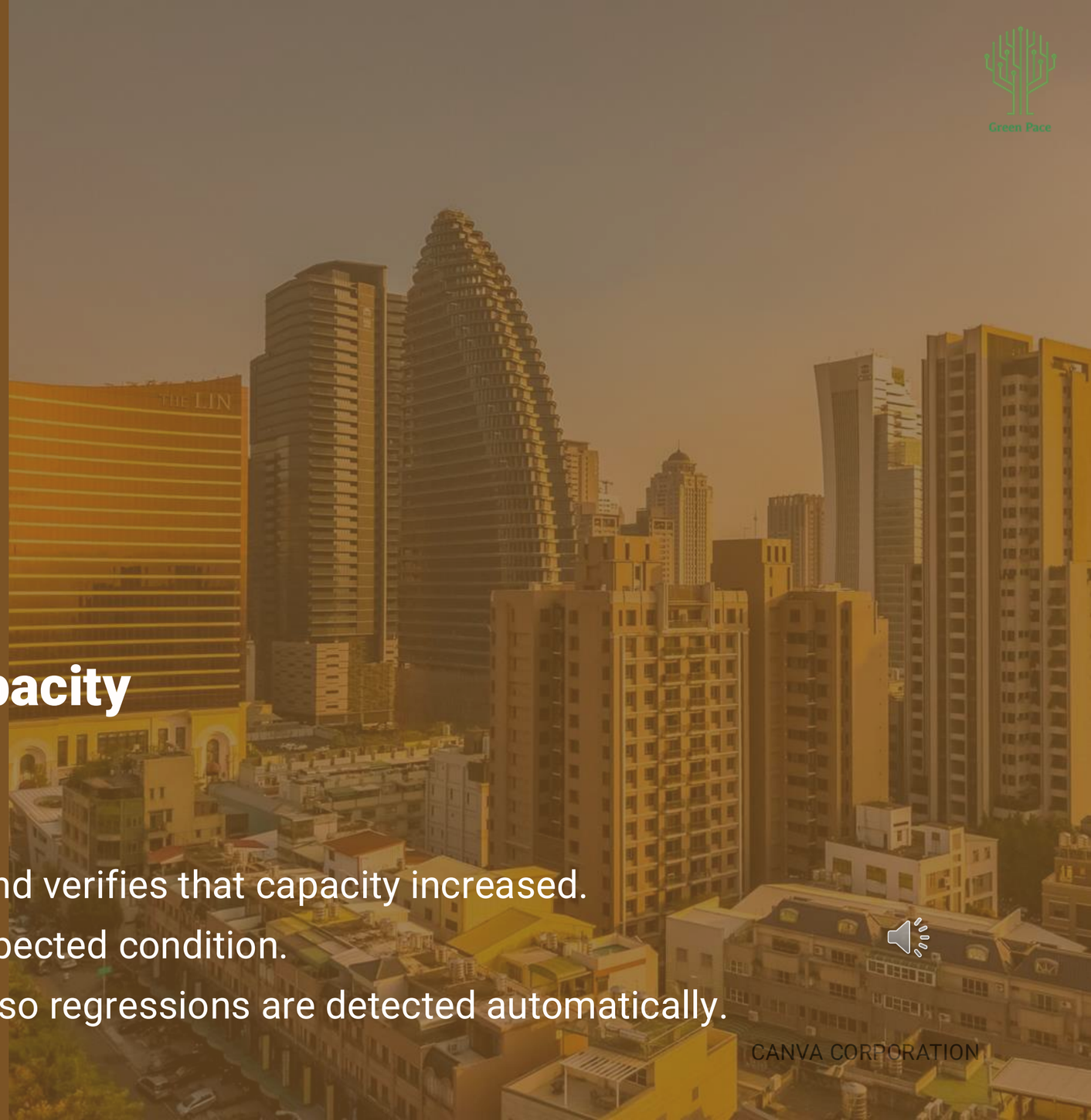
18

Unit Test:

ReverseIncreasesCollectionCapacity

Test Type: Positive test

- Reverses capacity without changing size and verifies that capacity increased.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.

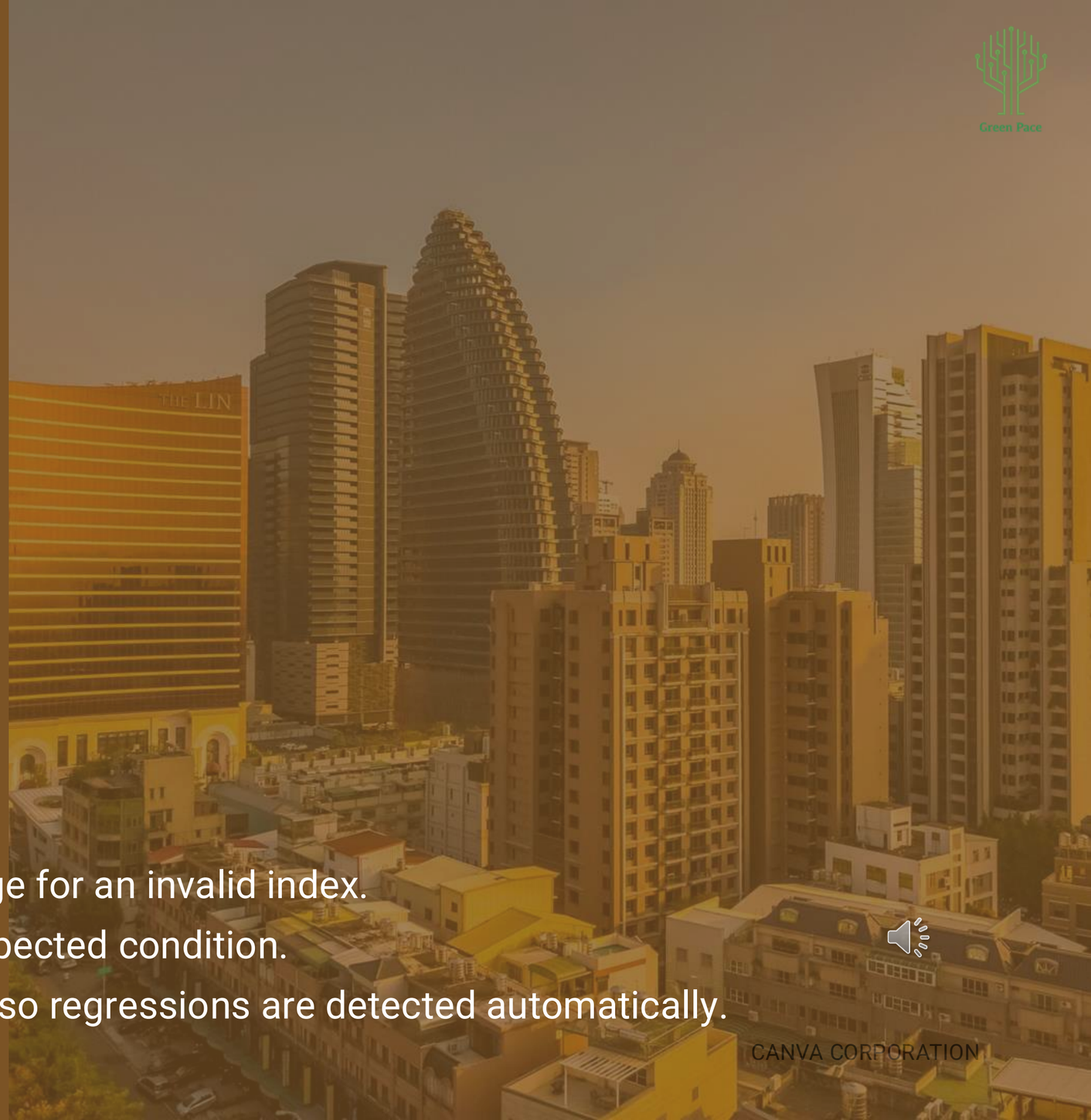


19

Unit Test: OutOfRangeExceptionThrown

Test Type: Negative test

- Confirms vector::at throws std::out_of_range for an invalid index.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



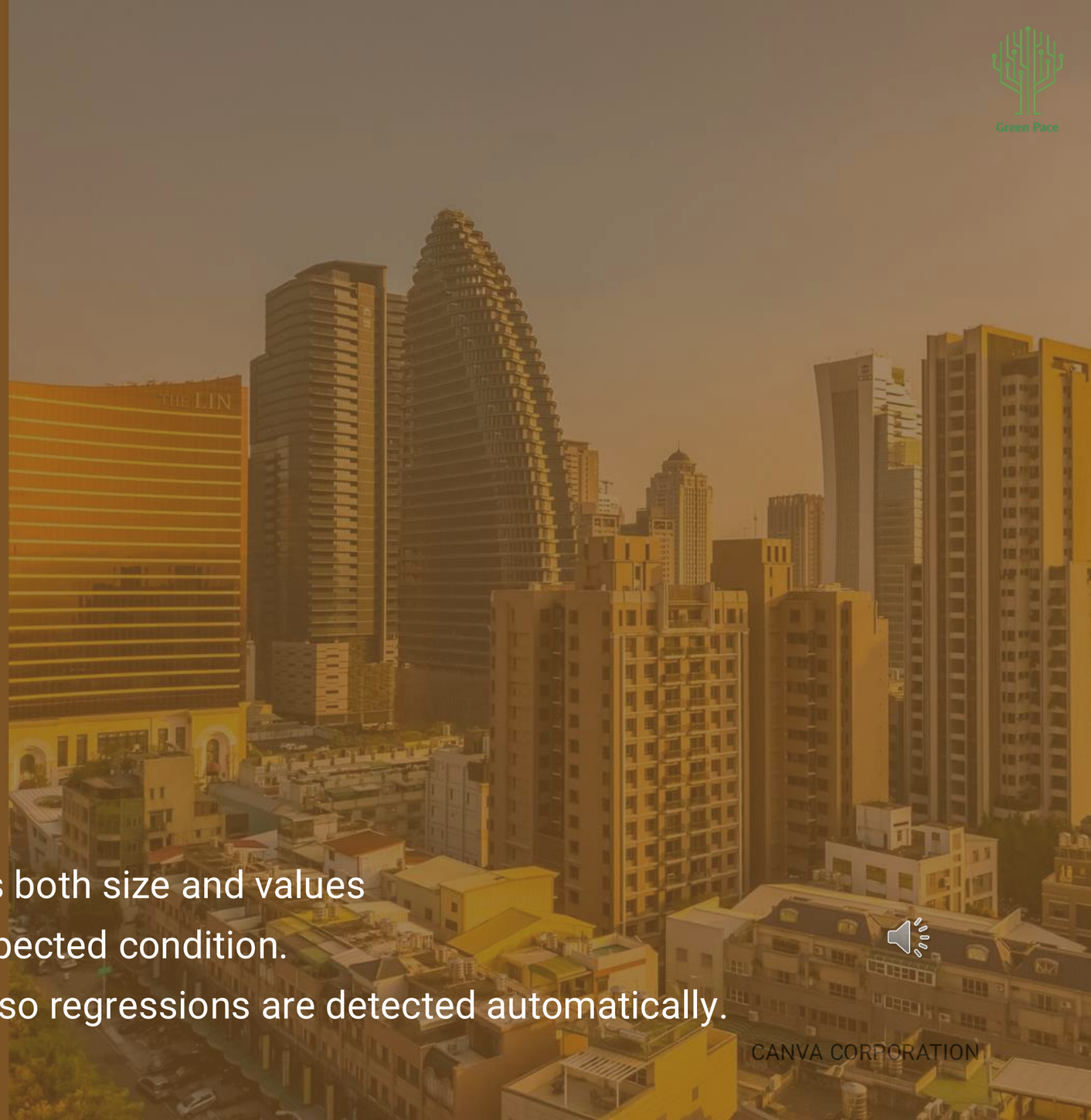
20

Unit Test:

AssignValuesToCollection

Test Type: Positive test

- Assigns five copies of value 10 and verifies both size and values
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.



Green Pace

21

Unit Test:

IncreaseCollectionReserveAboveMaxSize

Test Type: Negative test

- Confirms reserving beyond max_size throws std::length_error.
- Google Test macros explicitly prove the expected condition.
- Next step: retain this test in the CI pipeline so regressions are detected automatically.

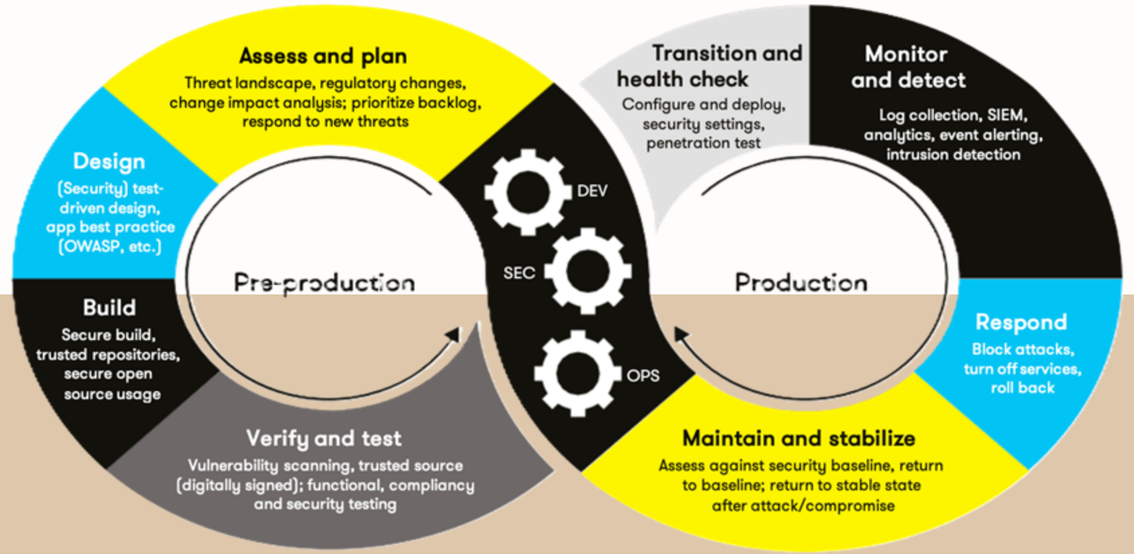
Unit Testing Results

In Google Suite, 21 tests were executed successfully.

Negative tests verify expected exceptions for invalid access and excessive reserve requests.

Positive tests verify normal vector behavior such as insertion, resizing, clearing, erasing, reserving, and assignment.

Unit testing provides a repeatable verification layer within the broader defense-in-depth strategy.



CANVA CORPORATION

Green Pace

DevSecOps Automation Pipeline

Plan

Threat modeling, attack-surface review, and security requirements.

Code

IDE/compiler warnings and secure coding guidance provide immediate feedback

Build/Verify

Compile with strong warning settings and run static analysis such as Cppcheck and Visual Studio Code Analysis

Test

Run unit, negative, integration, and security tests automatically.

Release

Enforce quality/security gates before deployment.

Operate

Monitor logs, security events, and application behavior; feed findings back into development.

Green Pace

Risks and Benefits

Waiting increases remediation cost because defects may reach production and require emergency fixes. Priority should favor vulnerabilities with the greatest potential impact, especially injection and memory-safe issues.

ACT NOW:

Reduces the likelihood that coding defects become exploitable vulnerabilities.

BENEFIT:

Automated checks scale with the development team and provide repeatable evidence for audits.

COST:

Tooling, configuration, false-positive review, and developer time are required.

Green Pace

Recommendations & Future Gaps

- Make security gates mandatory in pull requests and CI/CD builds.
- Track unresolved static-analysis findings and require documented risk acceptance for exceptions.
- Expand automated testing to include security-focused negative tests and regression tests.
- Review dependencies, compiler settings, static-analysis rules, and threat models regularly.
- Maintain policy change control: annual review plus updates when regulations, technology, or threats change.
- Continue developer training so secure practices remain consistent as the team grows.

Green Pace

Conclusion

- Green Pace’s policy converts implicit secure-development practices into repeatable standards.
- Defense in depth connects coding practices, encryption, Triple-A, static analysis, and unit testing.
- Security should be addressed throughout the software lifecycle, not left until the end.
- Continuous testing and automation provide earlier detection, lower remediation costs, and stronger audit evidence.
- The goal is not a single perfect security control; it is a resilient system of complementary controls.

Green Pace

References

Software Engineering Institute. (n.d.). SEI CERT C++ coding standard

Software Engineering Institute. (n.d.). SEI CERT C coding standard

Cppcheck. (n.d.). Cppcheck: A tool for static C/C++ code analysis

Microsoft. (n.d.). Code analysis for C/C++ overview. Microsoft Learn.

SonarSource. (n.d.). C/C++ analysis. SonarQube documentation.

Green Pace